Dynamics and Control of a Radiation-Zone Inspection Robot

Mathematical model, adaptive fuzzy-PID heading control, stability analysis and data connection

Dynamics and Control of Robotics

1 Problem and assumptions

A differential-drive ground robot must visit a list of inspection points inside a damaged reactor building. The floor has rubble patches where the wheels slip, falling debris knocks the robot sideways, and the heading sensor is noisy. The robot has no pre-drawn path; a planner computes the desired heading online and a controller makes the robot follow it.

Assumptions. Planar motion; rigid body; centre of mass at the midpoint of the wheel axle; rolling without lateral slip (non-holonomic constraint); viscous friction; wheel slip modelled by a traction ratio κ.

Symbols. (x, y) position, θ heading, v forward speed, ω yaw rate, m mass, I yaw inertia, r wheel radius, L track width, b, c translational and rotational viscous friction, τ_R, τ_L right and left wheel torques.

2 Kinematics

With $q = [x\ y\ \theta]^\top$ and $\eta = [v\ \omega]^\top$:

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \omega + d_w(t) \tag{1}$$

The no-side-slip constraint is $\dot{x}\sin\theta - \dot{y}\cos\theta = 0$. The term $d_w(t)$ is the yaw-rate kick from a debris strike (a pulse of ± 1.5 rad/s for 0.3 s).

3 Dynamics

3.1 Newton–Euler

Wheel torques produce a net traction force and a yaw moment:

$$F = \frac{\tau_R + \tau_L}{r}, \qquad M = \frac{L}{2r}\,(\tau_R - \tau_L) \tag{2}$$

$$m\,\dot{v} = F - b\,v, \qquad I\,\dot{\omega} = M - c\,\omega \tag{3}$$

Inverting the wheel map gives the torques needed for a given F and M:

$$\tau_R = \frac{rF}{2} + \frac{rM}{L}, \qquad \tau_L = \frac{rF}{2} - \frac{rM}{L} \tag{4}$$

3.2 Euler–Lagrange check

Kinetic energy $T = \frac{1}{2}m(\dot{x}^2 + \dot{y}^2) + \frac{1}{2}I\dot{\theta}^2$ with the constraint $A(q)\dot{q} = 0$, $A = [\sin\theta\ -\cos\theta\ 0]$. Writing $\dot{q} = S(q)\eta$ with $S = \begin{bmatrix} \cos\theta & 0 \\ \sin\theta & 0 \\ 0 & 1 \end{bmatrix}$ and eliminating the constraint force gives $S^\top M_q S\,\dot{\eta} + S^\top M_q \dot{S}\,\eta = S^\top B\tau$. With the centre of mass on the axle, $S^\top M_q \dot{S} = 0$, so there are no Coriolis terms and the result is exactly (3).

3.3 Inner velocity servos and slip

Each axis has an inner servo with friction feed-forward:

$$F = b\,v + K_v(\kappa v_{ref} - v), \qquad M = c\,\omega + K_\omega(\kappa\,\omega_{ref} - \omega), \qquad K_v = \frac{m}{\tau_v},\ K_\omega = \frac{I}{\tau_w} \tag{5}$$

Substituting into (3), the friction terms cancel and

$$\tau_v\,\dot{v} + v = \kappa\,v_{ref}, \qquad \tau_w\,\dot{\omega} + \omega = \kappa\,\omega_{ref} \tag{6}$$

$\kappa \in (0, 1]$ is the traction ratio: ground speed equals κ times the commanded wheel surface speed ($\kappa = 1$ on clear floor, $\kappa = 0.6$ on rubble). So rubble does not change the dynamics' time constant in this model; it removes 40% of the actuation gain. The controller output is ω_{ref}.

Parameters. $m = 15$ kg, $I = 0.70\ \mathrm{kg\,m^2}$, $r = 0.10$ m, $L = 0.40$ m, $b = 4\ \mathrm{N\,s/m}$, $c = 0.8\ \mathrm{N\,m\,s/rad}$, $\tau_v = 0.20$ s, $\tau_w = 0.15$ s, giving $K_v = 75\ \mathrm{N\,s/m}$ and $K_\omega = 4.67\ \mathrm{N\,m\,s/rad}$. $v_{ref} = 0.6$ m/s, $|\omega_{ref}| \le 2$ rad/s.

4 Dynamic route: planner

The map contains only obstacles that have come within the sensing range (3 m). With goal direction $u = (g-p)/\|g-p\|$, and for each mapped obstacle i with surface clearance $d_i = \|p-o_i\| - \rho_i - R_{robot}$, unit outward normal n_i and tangent t_i (oriented so $t_i \cdot u \ge 0$):

$$f = u + \sum_{d_i < D_0} K_{rep}\left(\frac{1}{d_i} - \frac{1}{D_0}\right)(0.6\,n_i + 0.8\,t_i), \qquad \psi_d = \mathrm{atan2}(f_y, f_x) \tag{7}$$

$D_0 = 1.5$ m, $K_{rep} = 0.45$. The heading error is $e = \mathrm{wrap}(\psi_d - \theta - \nu(t))$ where ν is sensor noise (amplitude 0.015 rad).

5 Controllers

5.1 Fixed PID

$$\omega_{ref} = \mathrm{sat}_{\pm 2}\Big(K_p e + K_i \int e\,dt + K_d\,\dot{e}_f\Big), \qquad K_p = 2.0,\ K_i = 0.15,\ K_d = 0.25 \tag{8}$$

The derivative uses a first-order filter ($T_f = 0.04$ s) and the integral is clamped to ± 0.5 (anti-windup).

5.2 Adaptive fuzzy-PID (gain scheduling)

The gains are scaled online: $K_p = K_{p0}s_p$, $K_i = K_{i0}s_i$, $K_d = K_{d0}s_d$. Inputs are normalised magnitudes $\varepsilon = |e|/0.6$ and $\delta = |\dot{e}_f|/1.5$ (clipped to $[0, 1]$), each with three triangular sets Small, Medium, Large:

$$\mu_S(u) = \max(0, 1 - 2u), \quad \mu_M(u) = \max(0, 1 - 2|u - \tfrac{1}{2}|), \quad \mu_L(u) = \max(0, 2u - 1) \tag{9}$$

Rule firing strength $w_{jk} = \mu_j(\varepsilon)\mu_k(\delta)$; the Sugeno output is $s = \sum_{j,k} w_{jk}\,c_{jk}$ with

$$c^{(p)} = \begin{bmatrix} 0.8 & 0.9 & 1.0 \\ 1.2 & 1.3 & 1.4 \\ 1.6 & 1.8 & 2.0 \end{bmatrix},\ c^{(d)} = \begin{bmatrix} 0.8 & 1.2 & 1.6 \\ 0.8 & 1.4 & 1.8 \\ 0.8 & 1.5 & 2.0 \end{bmatrix},\ s_i = \sum_j \mu_j(\varepsilon)\,[1,\,0.5,\,0]_j \tag{10}$$

(rows: ε Small/Medium/Large; columns: δ Small/Medium/Large). Reading: a big, fast-changing error raises K_p and K_d and switches K_i off to avoid windup; a small error keeps the gains low so the robot stays smooth.

6 Linearisation and stability

Linearising about straight motion and ignoring d_w and ν, from (1) and (6):

$$G(s) = \frac{\theta(s)}{\omega_{ref}(s)} = \frac{\kappa}{s\,(\tau_w s + 1)}, \qquad C(s) = K_p + \frac{K_i}{s} + \frac{K_d\, s}{T_f s + 1}, \qquad L(s) = C(s)G(s) \tag{11}$$

With an ideal derivative the characteristic polynomial is

$$\tau_w s^3 + (1 + \kappa K_d)\, s^2 + \kappa K_p\, s + \kappa K_i = 0 \tag{12}$$

Routh–Hurwitz. All coefficients positive and

$$(1 + \kappa K_d)\, \kappa K_p > \tau_w\, \kappa K_i \iff (1 + \kappa K_d) K_p > \tau_w K_i \tag{13}$$

For the base gains with $\kappa = 0.6$: $(1 + 0.15)(2.0) = 2.30 > 0.15 \times 0.15 = 0.0225$, stable with a very large margin.

Effect of slip and of the adaptation (full filtered loop, computed numerically):

Operating point	K_p, K_i, K_d	crossover	phase margin	closed-loop poles
Fixed, clear floor ($\kappa = 1$)	2.0, 0.15, 0.25	1.99 rad/s	85.0°	$-0.08, -1.97, -7.2, -22.5$
Fixed, rubble ($\kappa = 0.6$)	2.0, 0.15, 0.25	1.19 rad/s	84.7°	$-0.08, -1.14, -6.9, -23.5$
Adaptive, rubble, large fast error	3.6, 0, 0.375	2.14 rad/s	84.4°	$-2.45, -6.5, -22.8$
Adaptive, clear, small error	1.6, 0.15, 0.20	1.59 rad/s	84.4°	$-0.10, -1.53, -7.0, -23.0$

Interpretation: slip ($\kappa = 0.6$) scales the loop gain and drops the crossover frequency from 1.99 to 1.19 rad/s, so the robot responds more slowly exactly when the floor is worst. When the error is large the scheduler raises K_p and K_d by 1.8 and 1.5, which brings the crossover back to 2.14 rad/s (above the clear-floor value) with the phase margin unchanged at about 84°. When the error is small the gains drop, which keeps the response smooth. All 16 corners of the scheduling range ($\kappa \in \{0.6, 1\}$, $s_p, s_d \in \{0.8, 2.0\}$, $s_i \in \{0, 1\}$) have all poles in the left half plane.

Limitation. This is a frozen-gain check at each operating point. It does not by itself prove stability of the time-varying fuzzy law, which would need slowly varying gains or a Lyapunov argument. Stability of the nonlinear closed loop is demonstrated by simulation, not proved.

7 Radiation sensing model and data connection

7.1 Dose field

For hidden sources at s_k, the dose rate at position p is

$$D(p) = D_0 + \sum_k \frac{S}{1 + (\|p - s_k\|/w)^2}, \qquad D_0 = 0.1,\ S = 45,\ w = 0.8 \text{ m} \tag{14}$$

A hot zone is declared when $D > 12$. The estimated hotspot is the dose-weighted centroid of one run of consecutive samples above the threshold, $\hat{p} = \sum_i D_i p_i / \sum_i D_i$. This is biased toward the robot's path (the path passes about 0.6–1 m from each source), so a position error of that order is expected.

7.2 Actuator identification from data

From logged step tests (t_j, u_j, y_j) of commanded and measured yaw rate, fit $y(t) = \kappa u_0(1 - e^{-(t-t_0)/\tau_w})$ by least squares:

$$(\hat{\kappa}, \hat{\tau}_w) = \arg\min_{\kappa,\tau} \sum_j \left(y_j - \kappa u_0 (1 - e^{-(t_j - t_0)/\tau})\right)^2 \tag{15}$$

Then $K_\omega = I/\hat{\tau}_w$ follows from Section 3, and $\hat{\kappa}$ measured on a rubble surface is the slip ratio used in the model.

Important. The two data files in `matlab/data/` are *synthetic*: they were generated from the model plus noise so the pipeline runs out of the box. On those files the fit recovers $\hat{\kappa} = 1.00, \hat{\tau}_w = 0.152$ s (clear) and $\hat{\kappa} = 0.60, \hat{\tau}_w = 0.147$ s (rubble), which only confirms the code works, not that the model matches a real robot. To connect real data, log the same columns from the real robot or from a simulator such as Webots and replace the file; the scripts need no other change.

8 Results of the nonlinear mission simulation

Euler integration at 10 ms; all disturbances active; both controllers use the same base gains.

	Fixed PID	Adaptive fuzzy-PID
Mission time	47.4 s	47.3 s
Inspection points reached	5/5	5/5
Leaks found / obstacles mapped	3/3, 10/10	3/3, 10/10
Closest approach to rubble	21 cm	22 cm
Mean heading error	0.107 rad	0.099 rad (7% lower)
Peak heading error	0.91 rad	0.87 rad (4% lower)
Peak wheel torque	2.49 N m	2.49 N m

Both controllers complete the mission safely; the adaptive one is modestly better at recovering from hits. The size of the improvement depends on the chosen gains, disturbances and tables, so it should be quoted as a result of this setup, not as a general claim.

9 What is and is not modelled (planar mission, Sections 1–8)

Modelled: rigid-body dynamics, actuator servos, slip, impacts as yaw-rate kicks, sensor noise, online mapping and replanning, radiation field. The planar mission has no terrain height, no motor torque limits, no lidar map and a reactive planner that can get trapped in a dead end; Section 10 adds all of these.

10 Advanced extension: terrain, torque limits, lidar map, global planner

The planar mission above is left unchanged. A second scenario (`world_hard.m`: 25 obstacles, four height mounds, rubble patches and a U-shaped dead end with the last inspection point behind it) removes the main limitations.

10.1 3D terrain and gravity

The height map is $z(x,y) = \sum_k h_k \exp\!\big(-\|(x,y)-c_k\|^2/(2s_k^2)\big)$. With $\nabla z = (z_x, z_y)$, the slope along the heading and the tilt are

$$\sin\varphi = \frac{\nabla z \cdot (\cos\theta, \sin\theta)}{\sqrt{1 + (\nabla z \cdot (\cos\theta, \sin\theta))^2}}, \qquad \text{tilt} = \arctan\|\nabla z\| \tag{16}$$

Gravity enters the translational equation, and slope also reduces traction:

$$m\dot{v} = F - bv - mg\sin\varphi, \qquad \kappa_{\text{eff}} = \kappa\,(1 - 0.8\,\|\nabla z\|) \tag{17}$$

The speed servo gets a pitch feed-forward, $F = bv + mg\sin\varphi + K_v(\kappa v_{\text{ref}} - v)$, in the same way as the friction feed-forward.

10.2 Motor torque limits

The wheel torques from Eq. (4) are saturated, $|\tau_R|, |\tau_L| \le \tau_{\max}$, and F, M are recomputed from the limited torques with Eq. (2). When a limit is hit the servo can no longer cancel gravity or friction, so the speed and turn rate lag the command. Limits: $\tau_{\max} = 3$ N m (wheeled), 4 N m (tracked).

10.3 Lidar, occupancy grid and two-layer planner

A 2D lidar (90 rays, 4 m, 10 Hz) marks hit cells of a 0.2 m occupancy grid (static world, binary). Terrain cells steeper than the drive's climbable slope are also marked untraversable when within sensor range (a depth sensor is assumed). The grid is inflated by 0.65 m and A* (8-connected, no corner cutting, cost $1 + 8\|\nabla z\|$, unknown cells free) is re-run every 0.5 s. The point 1.5 m ahead on the path becomes the target of the reactive layer, which is Eq. (7) with repulsion from the nearest mapped cell (gain halved). The heading PID or adaptive fuzzy-PID is unchanged. With the global layer off, the target is the waypoint itself (reactive-only).

10.4 Drive type

`wheeled`: $\kappa_{\text{rubble}} = 0.6$, slope limit 0.20, $\tau_{\max} = 3$. `tracked`: $\kappa_{\text{rubble}} = 0.8$, slope limit 0.45, $\tau_{\max} = 4$. This is a parameter change only, not a track–terrain contact model.

10.5 Results

Drive	Planner	Outcome	Time [s]	Mean $\|e\|$ fixed / adaptive [rad]
wheeled	reactive-only	hits dead end, 3/4 points	45	0.384 / 0.385
wheeled	A* + reactive	4/4 points, 29–31 cm clear	74.4 / 74.5	0.217 / 0.210
tracked	reactive-only	hits dead end, 3/4 points	39	0.151 / 0.135
tracked	A* + reactive	4/4 points, 39 cm clear	64.2 / 64.5	0.194 / 0.183

Without the global layer the robot drives into the back wall of the U (it never stops, so the run ends as a collision); with A* it sees the wall, replans and goes around. Wheeled reactive-only also drives over the steep pile (22° tilt, below the 25° warning level), while A* avoids it. The tracked robot's shorter time comes from less slip and a higher torque limit. The adaptive controller's mean heading error is 3–6% lower with A*, the same size of effect as in Section 8.

10.6 Remaining limits

This is one fixed layout, not a statistical study. Both runs find 2 of 3 leaks because the replanned route passes more than 1.3 m from one source. Tilt is reported but there is no tip-over dynamics. Getting stuck is only represented through the slope and torque limits, not by wheel–debris contact. The lidar map is binary and assumes a static world with known pose, so it is not full SLAM. No real or Webots data are included; the CSV scripts accept real logs in the same format.

MATLAB files

`robot_params.m`	all parameters
`robot_accel.m`	Newton–Euler dynamics with servos
`simulate_mission.m`	nonlinear simulation, planner, sensing
`fuzzy_scales.m`	Sugeno fuzzy gain scheduler
`analyze_stability.m`	$G(s)$, Routh–Hurwitz, poles, Bode, step, root locus
`identify_actuator.m`	least-squares identification from CSV
`analyze_radiation.m`	hotspot estimation from CSV
`run_mission.m`, `compare_pid.m`	fixed vs adaptive plots and metrics
`adv_params.m`, `world_hard.m`	advanced scenario parameters and world
`simulate_advanced.m`	terrain, torque limits, lidar map, A* + reactive
`compare_advanced.m`	advanced scenario comparison
`demo_key_values.m`, `run_all.m`	demo and driver